



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

LAB PROJECT REPORT

FPGA Lab

TITLE: ATM MACHINE USING FPGA

Submitted by

Name: P Aryan

Reg no: 230959128

Name: Raghav Prasad

Reg no: 230959124

Name: K Rahul

Reg no: 230959126

Batch A2

Index

- Objective
- Introduction
- Project Theory Explanation
- Verilog Code
- Results
- Conclusion
- References

1. Objective

The primary objective of this project is to design and implement an ATM machine system using Field-Programmable Gate Array (FPGA) to demonstrate real-time banking transactions such as authentication, balance inquiry, deposits, and withdrawals. The system aims to simulate the core functionality of an ATM using digital logic design and finite state machines (FSMs) and ensure security features, such as PIN validation and restricted access, for ATM transactions.

2. Introduction

In this project, an **FPGA-based ATM machine** is designed and implemented using **digital logic circuits and finite state machines (FSMs)** to simulate core ATM functionalities. The system includes essential features such as user authentication, account balance management, cash transactions, and display interfaces for user interaction. By leveraging the reconfigurable nature of FPGAs, the design ensures **high performance**,

security, and scalability compared to traditional microcontroller-based ATM implementations.

3. Project Theory Explanation

This FPGA-based ATM system is designed to handle deposits and withdrawals, ensuring that transactions are valid and providing warnings for incorrect operations.

1. Inputs and User Interaction

The ATM machine uses switches (SW0 - SW5) to select the deposit or withdrawal amount.

Each switch corresponds to a specific denomination:

SW0 = \$1, SW1 = \$5, SW2 = \$20, SW3 = \$50, SW4 = \$100, Push

Buttons are used to perform different operations:

Top button → Deposit the selected amount

Bottom button → Withdraw the selected amount

Middle button → Reset the balance to zero

2. Processing the Transaction Switch Handling

A 4-to-1 MUX (Multiplexer) is used to determine which amount is selected for the transaction.

A decoder (3-to-8) is used to activate the correct output display for each amount.

Deposit Functionality

When the deposit button is pressed, the selected amount is added to the total balance.

If multiple switches are on at the same time, a warning LED (LED0) is turned on.

If the deposit exceeds the maximum balance (\$5000), another warning LED (LED1) turns on, preventing further deposits.

Withdrawal Functionality

When the withdraw button is pressed, the selected amount is subtracted from the total balance.

If the withdrawal exceeds the available balance, a warning LED (LED2) turns on, preventing the transaction.

3. Displaying the Balance

The balance is converted to BCD (Binary Coded Decimal) using a Binary-to-BCD converter.

A BCD-to-7-segment display module is used to show the balance on a 7-segment display.

4. Error Handling and Warnings

Multiple Switches Warning (LED0)

If more than one switch is ON at the same time, LED0 turns on, indicating an invalid selection.

Maximum Balance Warning (LED1) If the deposit exceeds \$5000, LED1 turns on, preventing further deposits.

Insufficient Balance Warning (LED2)

If the user tries to withdraw more than the available balance, LED2 turns on, preventing the transaction.

5. Reset Functionality

When the reset button (middle button) is pressed, the balance is set to zero, and all warning LEDs are turned off.

Final Outcome

The user can deposit money, withdraw money, and view their account balance on the 7-segment display.

The ATM prevents invalid operations using warning LEDs.

The project ensures safe and reliable transactions within the given balance constraints.

4. Verilog Code

```
module debounce (
    input clk,
    input rst,
    input button,
    output reg debounced_button
);
    reg [15:0] count;
    reg button_prev;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            count <= 0;
            debounced_button <= 0;
            button_prev <= 0;
        end else begin
            if (button == button_prev) begin
                if (count == 16'hFFFF)
                    debounced_button <= button;
                else
                    count <= count + 1;
            end else begin
                count <= 0;
            end
            button_prev <= button;
        end
    end
endmodule
```

```
module Binary_BCD_Converter (
    input [15:0] binary_in,
    output reg [19:0] bcd_out
);
    integer i;
    always @(binary_in) begin
        bcd_out = 0;
        for (i = 0; i < 16; i = i + 1) begin
            bcd_out = {bcd_out[18:0], binary_in[15-i]};
            if (bcd_out[3:0] > 4) bcd_out[3:0] = bcd_out[3:0] + 3;
            if (bcd_out[7:4] > 4) bcd_out[7:4] = bcd_out[7:4] + 3;
            if (bcd_out[11:8] > 4) bcd_out[11:8] = bcd_out[11:8] + 3;
            if (bcd_out[15:12] > 4) bcd_out[15:12] = bcd_out[15:12] + 3;
            if (bcd_out[19:16] > 4) bcd_out[19:16] = bcd_out[19:16] + 3;
        end
    end
endmodule
```

```
module four_to_one_mux (
    input [3:0] in0, in1, in2, in3,
    input [1:0] sel,
    output reg [3:0] out
);
```

```

always @(*) begin
    case (sel)
        2'b00: out = in0;
        2'b01: out = in1;
        2'b10: out = in2;
        2'b11: out = in3;
        default: out = 4'b0000;
    endcase
end
endmodule

module SlowClock_100Hz (
    input clk,
    input rst,
    output reg slow_clk
);
    reg [20:0] count;
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            count <= 0;
            slow_clk <= 0;
        end else if (count == 499999) begin
            count <= 0;
            slow_clk <= ~slow_clk;
        end else begin
            count <= count + 1;
        end
    end
end
endmodule

module eight_bit_counter (
    input clk, rst,
    output reg [1:0] count
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            count <= 0;
        else
            count <= count + 1;
        end
    end
endmodule

module decoder (
    input [2:0] in,
    output reg [7:0] out
);
    always @(*) begin
        out = 8'b00000001 << in;
    end
endmodule

module BCD_Seven_segment (
    input [3:0] bcd,
    output reg [6:0] seg
);
    always @(*) begin
        case (bcd)
            4'd0: seg = 7'b0111111;
            4'd1: seg = 7'b0000110;
            4'd2: seg = 7'b1011011;
            4'd3: seg = 7'b1001111;
            4'd4: seg = 7'b1100110;
            4'd5: seg = 7'b1101101;
            4'd6: seg = 7'b1111101;
            4'd7: seg = 7'b0000111;
            4'd8: seg = 7'b1111111;
            4'd9: seg = 7'b1101111;
            default: seg = 7'b0000000;
        endcase
    end
endmodule

module Deposit (
    input clk, rst, deposit,
    input [7:0] amount,
    output reg [15:0] balance,
    output reg warning_LED
);

```

```

parameter MAX_BALANCE = 16'd500;
always @(posedge clk or posedge rst) begin
    if (rst) begin
        balance <= 0;
        warning_LED <= 0;
    end else if (deposit) begin
        if (balance + amount > MAX_BALANCE)
            warning_LED <= 1;
        else begin
            balance <= balance + amount;
            warning_LED <= 0;
        end
    end
end
endmodule

```

```

module WithDraw (
    input clk, rst, withdraw,
    input [7:0] amount,
    output reg [15:0] balance,
    output reg warning_LED
);
always @(posedge clk or posedge rst) begin
    if (rst) begin
        balance <= 0;
        warning_LED <= 0;
    end else if (withdraw) begin
        if (balance < amount)
            warning_LED <= 1;
        else begin
            balance <= balance - amount;
            warning_LED <= 0;
        end
    end
end
endmodule

```

```

module ATM_System_Top(
    input clk,
    input rst,
    input deposit,
    input withdraw,
    input [4:0] switches, // Amount switches
    output [2:0] leds,    // Warning LEDs
    output [6:0] seg,     // Seven-segment segment outputs
    output [3:0] an       // Anode controls for digit select
);

    wire [15:0] balance;
    wire [19:0] bcd_balance;
    wire [1:0] digit_index;
    wire [3:0] selected_digit;
    wire [7:0] amount;

    // Decode switch values to actual dollar amounts
    assign amount = (switches[0] ? 8'd1 : 8'd0) +
        (switches[1] ? 8'd5 : 8'd0) +
        (switches[2] ? 8'd20 : 8'd0) +
        (switches[3] ? 8'd50 : 8'd0) +
        (switches[4] ? 8'd100 : 8'd0);

    // Clock divider for display refresh rate (e.g., ~1kHz)
    wire slow_clk;
    ClockDivider #(50000) clkdiv (
        .clk(clk),
        .rst(rst),
        .slow_clk(slow_clk)
    );

    // Digit index counter (2-bit for 4 digits)
    eight_bit_counter digit_counter (
        .clk(slow_clk),
        .rst(rst),
        .count(digit_index)
    );

    // ATM Core
    ATM atm_core (

```

```

        .clk(clk),
        .rst(rst),
        .deposit(deposit),
        .withdraw(withdraw),
        .amount(amount),
        .balance(balance),
        .leds(leds)
    );

// Binary to BCD Converter
Binary_BCD_Converter bcd_conv (
    .binary(balance),
    .bcd(bcd_balance)
);

// 4-to-1 Multiplexer to select digit
four_to_one_mux mux (
    .sel(digit_index),
    .in0(bcd_balance[3:0]),
    .in1(bcd_balance[7:4]),
    .in2(bcd_balance[11:8]),
    .in3(bcd_balance[15:12]),
    .out(selected_digit)
);

// BCD to 7-segment decoder
BCD_Seven_segment display_decoder (
    .bcd(selected_digit),
    .seven_seg(seg)
);

// Digit decoder (activates one anode at a time)
decoder_digit_decoder (
    .sel(digit_index),
    .an(an)
);
endmodule

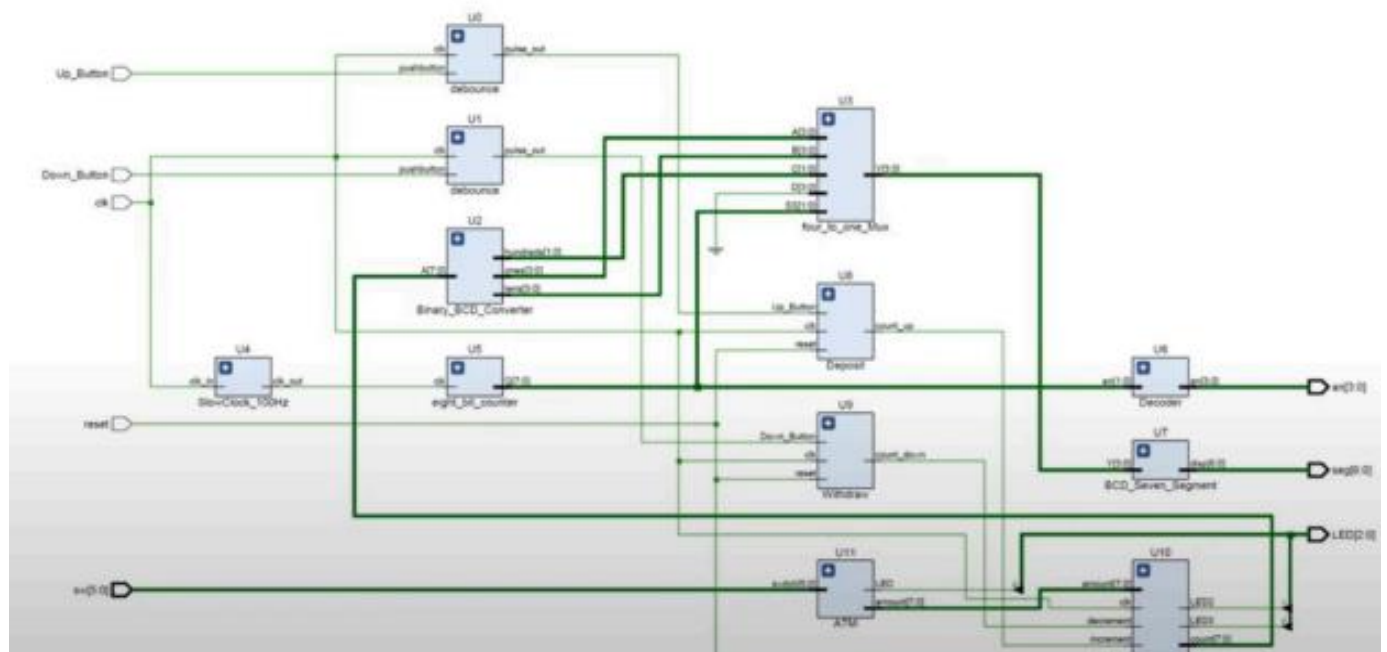
```

5.Result :- Waveform:



Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

CircuitDiagram:



6. Conclusion

This project successfully simulates an ATM machine using FPGA. This project not only highlights the feasibility of using FPGA in banking automation but also opens avenues for future enhancements, such as biometric authentication, RFID-based card transactions, and real-time encryption for secure communication. As financial security concerns continue to grow, FPGA-based designs offer a scalable and adaptable approach to modernizing banking technology.

7. References

- https://ijaem.net/issue_dcp/Design%20and%20Implementation%20of%20ATM%20Machine%20controller%20using%20VHDL.pdf
- <https://github.com/begumcelik/automated-teller-machine-design>
 - <http://www.kresttechnology.com/krest-academic-projects/krest-mtech-projects/ECE/M-TECH%20VLSI%202019-20/BASE%20PAPERS/28.p>

